

Infraestructura de Identidad Segura para la FIS

La arquitectura se basa en un modelo unificado de gestión de identidades de la FIS que integra un conjunto de middleware con un directorio OpenLDAP para almacenar los usuarios, grupos y directorios home bajo `dc=fis,dc=epn,dc=ec`; una PKI con una AC autofirmada con ECDSA para gestionar certificados de servidor para LDAP, cifrada con seguridad de la criptografía de curva elíptica; un realm Kerberos (FIS.EPN.EC) que proporciona Single Sign-On para los usuarios/servicio; y una capa de alta disponibilidad para minimizar interrupciones en el directorio y los servicios de autenticación. El host en WSL2 ejecuta el servidor LDAP primario (ldap1, puerto 1388) y el KDC primario. Dos contenedores Docker (ldap2 y kdc2) proporcionan la réplica LDAP secundaria y el KDC secundario, conectados mediante puertos mapeados (1389/1636 para ldap2, 2088/2749 para kdc2). HAProxy actúa como frontend para ambos nodos LDAP bajo `ldap.fis.epn.edu.ec`, y el servicio de Apache expone un endpoint protegido con Kerberos sobre TLS.

Generé una CA raíz (CN=Servidor CA Raiz) con OpenSSL, utilizando la curva `prime256v1`, firmada con ECDSA-SHA256 y válida por 10 años. Los certificados de servidor para los nodos LDAP y el servidor web asegurando una cadena basada únicamente en criptografía de curva elíptica. OpenLDAP configuré para escuchar en `ldaps` (636) y `ldap`, verificado con `openssl s_client` en el puerto 636 y revisando los campos `subject/issuer` con `openssl x509`. Apache utiliza TLS 1.3 en el puerto 443, usando un certificado emitido por la CA y asociado al FQDN `webserver.fis.epn.edu.ec`, que coincide con el certificado principal del servicio Kerberos utilizado para SPNEGO.

El DIT separa `ou=usuarios` y `ou=grupos` bajo la raíz de la FIS. Las entradas de usuario emplean los objetos `inetOrgPerson` y `posixAccount`, donde `inetOrgPerson` corresponde al esquema estándar de personas en LDAP y `posixAccount` donde se define atributos necesarios como `uidNumber`, `gidNumber`, directorio home y shell. El KDC lo implementé con MIT Kerberos para el realm FIS.EPN.EC, con 'principals' creados individualmente para cada usuario (`jperez`, `malvan`, `dnoboa`) y para los servicios (`HTTP/webserver.fis.epn.edu.ec`, `ldap/ldap1.fis.epn.edu.ec`). Para integrar Kerberos y LDAP, existen dos caminos principales. El primero es migrar Kerberos a una base de datos respaldada por LDAP, como hace FreeIPA, lo que simplifica la administración y disminuye el riesgo de inconsistencias. La segunda opción es importar el esquema LDAP de Kerberos en OpenLDAP y añadir el atributo `krbPrincipalName` a cada usuario. Esta última alternativa fue la que seleccioné. Esta opción ofrece mayor resiliencia, ya que la autenticación Kerberos opera de forma autónoma y permanece disponible si el backend LDAP falla, eliminando la dependencia directa entre ambos servicios. Sin embargo, pude observar que se necesita sincronización manual entre ambas bases de datos, lo que puede generar inconsistencias si las actualizaciones no se mantienen alineadas.

Utilice el servicio de Apache con `mod_auth_gssapi` restringido a Kerberos (`GssapiAllowedMech krb5`), protegiendo la ruta mediante SPNEGO. El SSO de extremo a extremo lo verifiqué desde un cliente Linux (`curl -negotiate`) y desde un cliente Windows con MIT Kerberos for Windows. Tras obtener un TGT, Firefox accedió a la página protegida y recibió una respuesta 200 sin solicitar credenciales, cumpliendo así el flujo Browser - Kerberos Ticket - Web Service.

Para replicar LDAP, utilicé `syncrepl` en modo `refreshAndPersist`. El servidor principal ejecuta el overlay `syncprov` y utiliza una cuenta dedicada llamada `replicator`, que solo tiene permisos de lectura. La réplica se mantiene actualizada recibiendo los cambios cada 10 segundos y se autentica de forma sencilla mediante un `bind simple`. Verifiqué que ambos nodos mostraban un `contextCSN` idéntico después de realizar una escritura en el principal, y pude comprobar que las lecturas desde la réplica siguieron funcionando normalmente incluso cuando apagué el servicio `slapd` en el host principal.

Para que Kerberos siempre estuviera disponible, configuré dos servidores: uno principal y uno secundario. El secundario recibe una copia exacta de la base de datos usando `kprop` y `kproxd`, autenticándose con las credenciales de los hosts (`host/krb5.fis.epn.edu.ec` y `host/kdc2.fis.epn.edu.ec`) que saco del `keytab`. El cambio automático (failover) es manual: cuando apago el principal, los clientes que tienen configurado el secundario

pueden seguir obteniendo tickets válidos. Durante las pruebas, para que el cambio fuera realmente automático, tuve que listar ambos servidores en el archivo 'krb5.conf'. Sin embargo, cada vez que reiniciaba la terminal, debía volver a configurar esta parte.

El balanceo de carga utilicé HAProxy en modo TCP en ambos nodos LDAP, expuesto como 'ldap.fis.epn.edu.ec' en el puerto 389. Traslade el host al puerto 1388 para liberar el puerto estándar para el balanceador. Los chequeos de salud se realizan cada dos segundos, con un umbral de dos fallos antes de marcar un backend como caído. La página de estadísticas de HAProxy y su integración con Prometheus me permitieron monitorear en tiempo real el estado de los backends.

En las pruebas realizadas, cuando agregué una nueva entrada en el maestro, la información apareció en la réplica en menos de 10 segundos. Los identificadores contextCSN coincidieron en ambos servidores, lo que demuestra que la replicación funcionó correctamente. Al probar el balanceo de carga, si detenía slapd en el maestro, HAProxy detectaba el problema y en unos 4 segundos redirigía automáticamente las consultas a la réplica. Esto también ocurrió cuando simulé una caída inesperada; la recuperación fue igual de rápida. En las pruebas de una partición de red, el sistema tardó unos 13 segundos en detectar el fallo, lo cual dependía del tiempo configurado para los chequeos de salud.

En cuanto a la alta disponibilidad de Kerberos, comprobé que, si el servidor principal dejaba de funcionar, el cambio al KDC secundario se realizaba en alrededor de 3,3 segundos y los usuarios seguían pudiendo autenticarse sin problema. Las pruebas de expiración de certificados también funcionaron: tanto curl como Firefox rechazaron de inmediato conexiones con certificados vencidos, mostrando alertas claras de TLS. El monitoreo con Prometheus y Grafana me permitió ver en tiempo real el uso de CPU, memoria y el estado de todos los servidores, y gracias a un script propio pude visualizar fácilmente si existía algún retraso en la replicación de LDAP.

Durante la implementación identifiqué dos problemas de centralización de nombres de host, que fueron los principales desafíos en la integración de Kerberos. En Linux, la resolución incorrecta del hostname de WSL a un dominio no previsto causaba errores en la construcción del principal de Kerberos e impedía la propagación adecuada de la base de datos. Lo solucioné desactivando la resolución inversa de DNS y asignando una entrada dedicada en '/etc/hosts'. En Windows, una configuración incorrecta del archivo hosts y una mala migración del archivo krb5.init me generó solicitudes de tickets Kerberos para FQDNs incorrectos y dificultaban la autenticación. Separar cada nombre de host en líneas individuales resolvió el problema.

Otra fuente de fallos en la negociación SPNEGO fue que Firefox utilizaba por defecto la pila SSPI nativa de Windows, que caía silenciosamente a NTLM después de que MIT Kerberos for Windows emitía un ticket. Como Apache estaba restringido a 'GssapiAllowedMech krb5', estos intentos eran rechazados con un 401 genérico. Deshabilitar 'network.auth.usesspi' y 'network.negotiate-auth.using-native-gsslib' y apuntar 'network.negotiate-auth.gsslib' directamente a la DLL de MIT Kerberos, forzó a Firefox a usar tokens Kerberos reales.

Tuve que importar manualmente el esquema LDAP de Kerberos tanto en el host principal como en la réplica, porque si lo hacía solo en el host, syncrpl no aceptaba la entrada de jperez y mostraba un error de sintaxis (krbPrincipalAux) que solo se veía al depurar kdc2. Además, cada vez que reiniciaba WSL, el archivo /etc/hosts se restablecía automáticamente y, a veces, eso dejaba un proceso slapd "zombie" en la réplica Docker: el sistema decía que estaba activo, pero en realidad no tenía el puerto ocupado. Para solucionar esto, tenía que terminar el proceso manualmente antes de que la réplica funcionara bien. También tuve un problema al probar la expiración de certificados, porque generé una nueva clave para el certificado de prueba en lugar de reutilizar la clave original, lo que rompió la configuración de TLS en Apache. Cuando volví a generar la solicitud (CSR) usando la clave correcta, el problema se resolvió.

Repositorio: <https://github.com/DJoel20/fis-miniidm/tree/main>